Deep Learning

Optimization for Training Deep Models

Lecturer: Duc Dung Nguyen, PhD.
Contact: nddung@hcmut.edu.vn

Faculty of Computer Science and Engineering
Hochiminh city University of Technology

Contents

Learning vs. Pure Optimization

Learning vs. Pure Optimization

- Machine learning usually acts indirectly
- Typically, the cost function can be written as an average over the training set, such as

$$J(\theta) = \mathbb{E}_{(x,y)\approx\hat{p}_{\text{data}}} L(f(x;\theta), y), \tag{1}$$

  where L is the **per-example loss function**, $f(x;\theta)$ is the predicted output when the input is x, and $\hat{p}_{data}$ is the empirical distribution. In the supervised learning case, y is the target output.
- $\rightarrow$ Objective function with respect to the training set

1. Empirical Risk Minimization

- The goal of a ML algorithm is to reduce the *expected generalization error* $\rightarrow$ **risk**.
- If we knew the true distribution $p_{\text{data}}(x, y)$, risk minimization would be an optimization task solvable by an optimization algorithm
- We only have a training set of samples $\rightarrow$ we have a machine learning problem

Learning vs. Pure Optimization

- Minimize the expected loss on the training set
- Minimize the *empirical risk*

$$\mathbb{E}_{(x,y)\approx\hat{p}_{\text{data}}}[L(f(x;\theta),y)] = \frac{1}{m}\sum_{i=1}^{m} L(f(x^{(i)};\theta),y^{(i)}) \tag{2}$$

  where m is the number of training examples.
- DL: rarely use empirical risk minimization!

2. Surrogate Loss Functions and Early Stopping

- Sometimes, the loss function is not one that can be optimized efficiently
- Optimizes a *surrogate loss function*: acts as a proxy but has advantages
- Minimizes a surrogate loss function but halts when a convergence criterion based on early stopping is satisfied

3. Batch and Minibatch Algorithms

- *Objective function* usually decomposes as a sum over the training examples
- Optimization: update the parameters based on an expected value of the cost function estimated *using only a subset* of the full cost function
- **Batch or deterministic gradient methods**: process all of the training examples simultaneously in a large batch
- **Stochastic** (or **online methods**): use only a single example at a time
- Most algorithms used for DL fall somewhere in between, using more than one but less than all of the training examples

Learning vs. Pure Optimization

Minibatch sizes:

- **Larger batches**: provide a more accurate estimate of the gradient, but *with less than linear returns*.
- Multicore architectures are usually underutilized by extremely small batches
- Using some absolute minimum batch size: no reduction in the time to process a minibatch

Learning vs. Pure Optimization

- Batch size defines performance
- Small batch size vs. regularizing $\rightarrow$ tradeoff?
- Mini bacthes should be selected randomly!

Challenges in Neural Network Optimization

Ill-Conditioning

- Causing SGD to get "stuck": even very small steps increase the cost function
- Given the Taylor expansion of cost function:

$$f(\mathbf{x}^{(0)} - \epsilon\mathbf{g}) \approx f(\mathbf{x}^{(0)}) - \epsilon\mathbf{g}^\top\mathbf{g} + \frac{1}{2}\epsilon^2\mathbf{g}^\top\mathbf{H}\mathbf{g} \tag{3}$$

a gradient descent step $-\epsilon\mathbf{g}$ will add to the cost:

$$\frac{1}{2}\epsilon^2\mathbf{g}^\top\mathbf{H}\mathbf{g} - \epsilon\mathbf{g}^\top\mathbf{g} \tag{4}$$

Local Minima

- Convex optimization: local minima = global minima
- What about **flat area**?
- We have reached a good solution if we find a **critical point** of any kind
- Non-convex functions (NN): **many** local minima

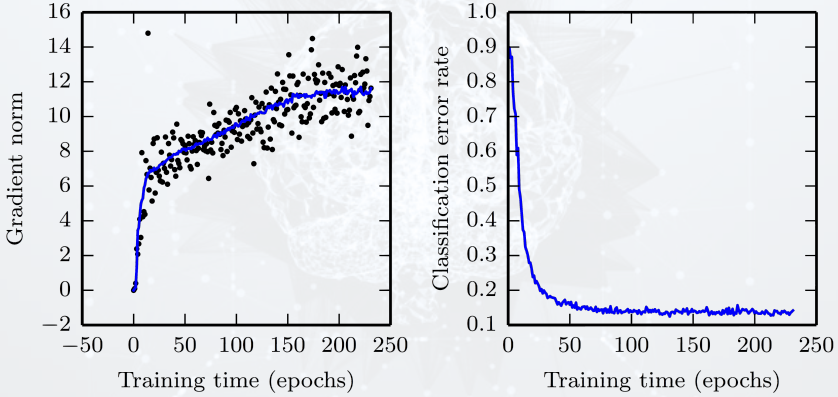
Local Minima

- **Model identifiability problem**: a sufficiently large training set can rule out all but one setting of the model's parameters
- Models with latent variables are often not identifiable (e.g. **weight space symmetry**)

Local minima

- Local minima can be problematic if they have high cost in comparison to the global minimum
- Serious problem for gradient-based optimization algorithms

Local minima

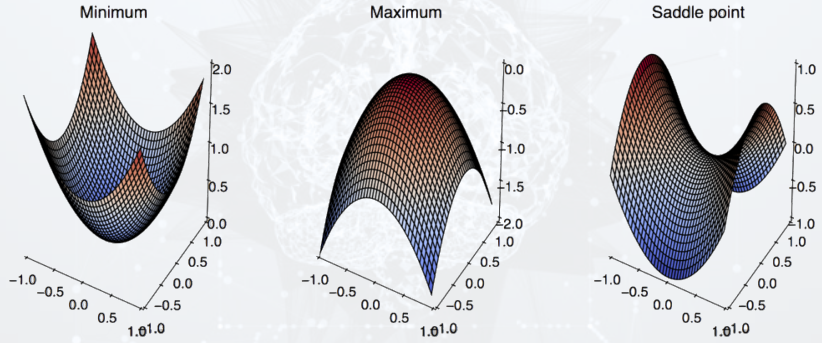


Plateaus, Saddle Points and Other Flat Regions

- **Saddle point**: point with zero gradient
  - A local minimum along one cross-section of the cost function and a local maximum along another cross-section
- In low-dimensional spaces: local minima are common
- In higher dimensional spaces: local minima are rare and saddle points are more common

Plateaus, Saddle Points and Other Flat Regions

Zero gradient, and Hessian with...



All positive eigenvalues All negative eigenvalues Some positive and some negative

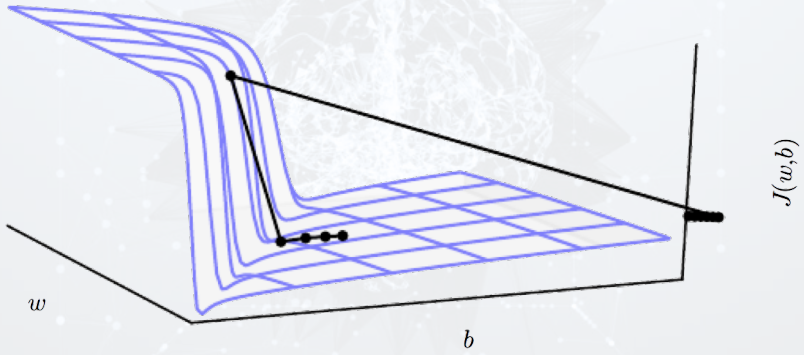
Plateaus, Saddle Points and Other Flat Regions

- Important properties of many random functions: **eigenvalues** of the **Hessian** become more likely to be **positive** as we reach regions of lower cost
- Critical points with **high cost** are far more likely to be saddle points
- Critical points with **extremely high cost** are more likely to be local maxima
- Gradient descent empirically seems to be able to *escape saddle points in many cases*

Cliffs and Exploding Gradients

- NN with many layers: often have extremely steep regions resembling **cliffs**
- Reason: multiplication of several large weights together
- The gradient update step *can move the parameters extremely far*, usually **jumping off of the cliff structure** altogether

Cliffs and Exploding Gradients



Cliffs and Exploding Gradients

- **Cliff structures** are most common in the cost functions for **RNN**
- Involve a multiplication of many factors, with one factor for each time step
- *Long temporal sequences* $\rightarrow$ extreme amount of multiplication

Basic Algorithms

Stochastic Gradient Descent (SGD)

Require: Learning rate schedule $\epsilon_1, \epsilon_2, \ldots$

Require: Initial parameter $\boldsymbol{\theta}$

$k \leftarrow 1$

while stopping criterion not met **do**

Sample a minibatch of m examples from the training set $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ with corresponding targets $\boldsymbol{y}^{(i)}$.

Compute gradient estimate: $\hat{\boldsymbol{g}} \leftarrow \frac{1}{m} \nabla_{\boldsymbol{\theta}} \sum_i L(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), \boldsymbol{y}^{(i)})$

Apply update: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \epsilon_k \hat{\boldsymbol{g}}$

$k \leftarrow k + 1$

end while

Stochastic Gradient Descent (SGD)

- SGD gradient estimator introduces a source of noise: the random sampling of m training examples
  - *Does not vanish!*
- Crucial parameter: **learning rate** ϵ
  - $\rightarrow$ Should be adapted over time
- A sufficient condition to guarantee convergence

$$\sum_{k=1}^{\infty} \epsilon_k = \infty, \text{ and } \sum_{k=1}^{\infty} \epsilon_k^2 < \infty \tag{5}$$

Stochastic Gradient Descent (SGD)

- Common practice: decay the learning rate linearly until iteration τ

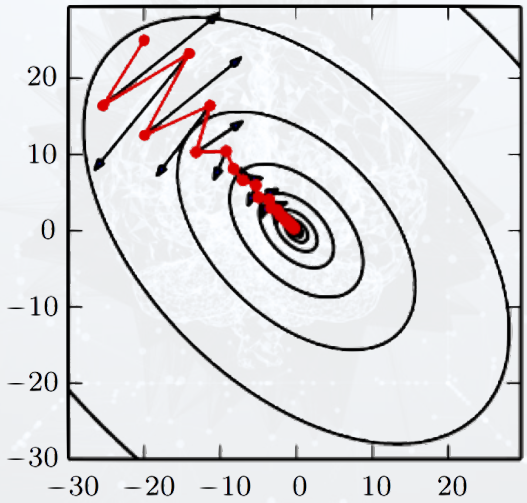
$$\epsilon_k = (1 - \alpha)\epsilon_0 + \alpha\epsilon_\tau \tag{6}$$

  with $\alpha = \frac{k}{\tau}$
- After iteration τ, leave ϵ constant
- Computation time per update does not grow with the number of training examples

Momentum

- Accelerate learning: high curvature, small but consistent gradients, or noisy gradients
- **Accumulate** an exponentially *decaying moving average of past gradients* and *continues to move in their direction*

Momentum



Momentum

Require: Learning rate ϵ, momentum parameter α
Require: Initial parameter $\boldsymbol{\theta}$, initial velocity $\boldsymbol{v}$
while stopping criterion not met **do**
Sample a minibatch of m examples from the training set $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ with corresponding targets $\boldsymbol{y}^{(i)}$.
Compute gradient estimate: $\boldsymbol{g} \leftarrow \frac{1}{m} \nabla_{\boldsymbol{\theta}} \sum_i L(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), \boldsymbol{y}^{(i)})$.
Compute velocity update: $\boldsymbol{v} \leftarrow \alpha \boldsymbol{v} - \epsilon \boldsymbol{g}$.
Apply update: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \boldsymbol{v}$.
end while

Nesterov Momentum

Require: Learning rate ϵ, momentum parameter α
Require: Initial parameter $\boldsymbol{\theta}$, initial velocity $\boldsymbol{v}$
while stopping criterion not met **do**
Sample a minibatch of m examples from the training set $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ with corresponding labels $\boldsymbol{y}^{(i)}$.
Apply interim update: $\tilde{\boldsymbol{\theta}} \leftarrow \boldsymbol{\theta} + \alpha \boldsymbol{v}$.
Compute gradient (at interim point): $\boldsymbol{g} \leftarrow \frac{1}{m} \nabla_{\tilde{\boldsymbol{\theta}}} \sum_i L(f(\boldsymbol{x}^{(i)}; \tilde{\boldsymbol{\theta}}), \boldsymbol{y}^{(i)})$.
Compute velocity update: $\boldsymbol{v} \leftarrow \alpha \boldsymbol{v} - \epsilon \boldsymbol{g}$.
Apply update: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \boldsymbol{v}$.
end while

Parameter Initialization Strategies

Parameter Initialization Strategies

- Modern initialization strategies: **simple** and **heuristic**
- The initial parameters need to **"break symmetry"** between different units
- Set the **biases** for each unit to **heuristically chosen constants**
- Extra parameters (e.g. parameters encoding the conditional variance of a prediction) are usually set to heuristically chosen constants

Parameter Initialization Strategies

- Initialize all the weights in the model to values drawn randomly from a **Gaussian** or **uniform distribution**
- The scale of the initial distribution has a large effect
  - The outcome of the optimization procedure
  - The ability of the network to generalize

Parameter Initialization Strategies

- Larger initial weights $\rightarrow$ stronger symmetry breaking effect, helping to avoid redundant units
- Too large initial weights $\rightarrow$ exploding values during forward propagation or back-propagation
- How to choose?
  - Optimization: the weights should be large enough to propagate information successfully
  - Regularization: encourage making the weights smaller

Choosing bias

- If a bias is for an output unit
  - Initialize the bias to obtain the right marginal statistics of the output
  - Assume that the initial weights are small enough that the output of the unit is determined only by the bias
  - Set the bias to the inverse of the activation function applied to the marginal statistics of the output in the training set
- Sometimes we may want to choose the bias to avoid causing too much saturation at initialization
  - When a unit controls whether other units are able to participate in a function

Algorithms with Adaptive Learning Rates

AdaGrad

- Individually adapts the learning rates of all model parameters: scaling them inversely proportional to the square root of the sum of all of their historical squared values
- Decreasing rate depends on partial derivative of parameter
- The net effect is greater progress in the more gently sloped directions of parameter space

AdaGrad

Require: Global learning rate ϵ
Require: Initial parameter $\boldsymbol{\theta}$
Require: Small constant δ, perhaps 10^{-7}, for numerical stability
Initialize gradient accumulation variable $\boldsymbol{r} = \mathbf{0}$
while stopping criterion not met **do**
Sample a minibatch of m examples from the training set $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ with corresponding targets $\boldsymbol{y}^{(i)}$.
Compute gradient: $\boldsymbol{g} \leftarrow \frac{1}{m} \nabla_{\boldsymbol{\theta}} \sum_i L(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), \boldsymbol{y}^{(i)})$.
Accumulate squared gradient: $\boldsymbol{r} \leftarrow \boldsymbol{r} + \boldsymbol{g} \odot \boldsymbol{g}$.
Compute update: $\Delta \boldsymbol{\theta} \leftarrow -\frac{\epsilon}{\delta + \sqrt{\boldsymbol{r}}} \odot \boldsymbol{g}$. (Division and square root applied element-wise)
Apply update: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \Delta \boldsymbol{\theta}$.
end while

RMSProp

- Modify AdaGrad to perform better in the non-convex setting
- Change the gradient accumulation into an exponentially weighted moving average
- Compared to AdaGrad: introduces a new hyperparameter, ρ, that controls the length scale of the moving average

RMSProp

Require: Global learning rate ϵ, decay rate ρ
Require: Initial parameter $\boldsymbol{\theta}$
Require: Small constant δ, usually 10^{-6}, used to stabilize division by small numbers
Initialize accumulation variables $\boldsymbol{r} = 0$
while stopping criterion not met **do**
Sample a minibatch of m examples from the training set $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ with corresponding targets $\boldsymbol{y}^{(i)}$.
Compute gradient: $\boldsymbol{g} \leftarrow \frac{1}{m} \nabla_{\boldsymbol{\theta}} \sum_i L(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), \boldsymbol{y}^{(i)})$.
Accumulate squared gradient: $\boldsymbol{r} \leftarrow \rho \boldsymbol{r} + (1 - \rho) \boldsymbol{g} \odot \boldsymbol{g}$.
Compute parameter update: $\Delta \boldsymbol{\theta} = -\frac{\epsilon}{\sqrt{\delta + \boldsymbol{r}}} \odot \boldsymbol{g}$. ($\frac{1}{\sqrt{\delta + \boldsymbol{r}}}$ applied element-wise)
Apply update: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \Delta \boldsymbol{\theta}$.
end while

Adam

- "**Adam**" derives from the phrase "adaptive moments"
- A variant on the combination of RMSProp and momentum with a few important distinctions

Adam

Require: Step size ϵ (Suggested default: 0.001)
Require: Exponential decay rates for moment estimates, ρ_1 and ρ_2 in $[0, 1)$. (Suggested defaults: 0.9 and 0.999 respectively)
Require: Small constant δ used for numerical stabilization (Suggested default: 10^{-8})
Require: Initial parameters $\boldsymbol{\theta}$
  Initialize 1st and 2nd moment variables $\boldsymbol{s} = \boldsymbol{0}$, $\boldsymbol{r} = \boldsymbol{0}$
  Initialize time step $t = 0$
  while stopping criterion not met **do**
    Sample a minibatch of m examples from the training set $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ with corresponding targets $\boldsymbol{y}^{(i)}$.
    Compute gradient: $\boldsymbol{g} \leftarrow \frac{1}{m} \nabla_{\boldsymbol{\theta}} \sum_i L(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), \boldsymbol{y}^{(i)})$
    $t \leftarrow t + 1$
    Update biased first moment estimate: $\boldsymbol{s} \leftarrow \rho_1 \boldsymbol{s} + (1 - \rho_1)\boldsymbol{g}$
    Update biased second moment estimate: $\boldsymbol{r} \leftarrow \rho_2 \boldsymbol{r} + (1 - \rho_2)\boldsymbol{g} \odot \boldsymbol{g}$
    Correct bias in first moment: $\hat{\boldsymbol{s}} \leftarrow \frac{\boldsymbol{s}}{1 - \rho_1^t}$
    Correct bias in second moment: $\hat{\boldsymbol{r}} \leftarrow \frac{\boldsymbol{r}}{1 - \rho_2^t}$
    Compute update: $\Delta\boldsymbol{\theta} = -\epsilon \frac{\hat{\boldsymbol{s}}}{\sqrt{\hat{\boldsymbol{r}}} + \delta}$ (operations applied element-wise)
    Apply update: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \Delta\boldsymbol{\theta}$
  end while

Adam

- **RMSProp**: lacks the correction factor $\rightarrow$ second-order moment estimate may have high bias early in training
- **Adam**: fairly robust to the choice of hyperparameters

Choosing the Right Optimization Algorithm

- **Which algorithm should one choose?**
- SGD, SGD with momentum, RMSProp, RMSProp with momentum, AdaDelta, Adam
- Your choice is depend largely on the familiarity with the algorithm (for ease of hyperparameter tuning)

Approximate Second-Order Methods

Newton's Method

- An optimization scheme based on using a second-order Taylor series expansion to approximate $J(\theta)$ near some point θ_0

$$J(\theta) \approx J(\theta_0) + (\theta - \theta_0)^\top \Delta_\theta J(\theta_0) + \frac{1}{2}(\theta - \theta_0)^\top H(\theta - \theta_0), \tag{7}$$

  where H is the Hessian of J with respect to θ evaluated at θ_0

- Newton parameter update rule:

$$\theta^* = \theta_0 - H^{-1}\Delta_\theta J(\theta_0) \tag{8}$$

Newton's Method

Require: Initial parameter $\boldsymbol{\theta}_0$
Require: Training set of m examples
  while stopping criterion not met **do**
    Compute gradient: $\boldsymbol{g} \leftarrow \frac{1}{m} \nabla_{\boldsymbol{\theta}} \sum_i L(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), \boldsymbol{y}^{(i)})$
    Compute Hessian: $\boldsymbol{H} \leftarrow \frac{1}{m} \nabla^2_{\boldsymbol{\theta}} \sum_i L(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), \boldsymbol{y}^{(i)})$
    Compute Hessian inverse: $\boldsymbol{H}^{-1}$
    Compute update: $\Delta\boldsymbol{\theta} = -\boldsymbol{H}^{-1}\boldsymbol{g}$
    Apply update: $\boldsymbol{\theta} = \boldsymbol{\theta} + \Delta\boldsymbol{\theta}$
  end while

Conjugate Gradients

- Efficiently avoid the calculation of the inverse Hessian by iteratively descending **conjugate directions**
- Seek to find a search direction that is **conjugate to the previous line search direction**, i.e. it will not undo progress made in that direction
- At training iteration t, the next search direction d_t takes the form:

$$d_t = \Delta\theta J(\theta) + \beta_t d_{t-1} \tag{9}$$

- β_t is a coefficient whose magnitude controls how much of the direction, d_{t-1}, we should add back to the current search direction

Conjugate Gradients

Require: Initial parameters $\boldsymbol{\theta}_0$
Require: Training set of m examples
  Initialize $\boldsymbol{\rho}_0 = \mathbf{0}$
  Initialize $g_0 = 0$
  Initialize $t = 1$
  while stopping criterion not met **do**
    Initialize the gradient $\boldsymbol{g}_t = \mathbf{0}$
    Compute gradient: $\boldsymbol{g}_t \leftarrow \frac{1}{m} \nabla_{\boldsymbol{\theta}} \sum_i L(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), \boldsymbol{y}^{(i)})$
    Compute $\beta_t = \frac{(\boldsymbol{g}_t - \boldsymbol{g}_{t-1})^\top \boldsymbol{g}_t}{\boldsymbol{g}_{t-1}^\top \boldsymbol{g}_{t-1}}$ (Polak-Ribière)
    (Nonlinear conjugate gradient: optionally reset β_t to zero, for example if t is a multiple of some constant k, such as $k = 5$)
    Compute search direction: $\boldsymbol{\rho}_t = -\boldsymbol{g}_t + \beta_t \boldsymbol{\rho}_{t-1}$
    Perform line search to find: $\epsilon^* = \text{argmin}_\epsilon \frac{1}{m} \sum_{i=1}^{m} L(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}_t + \epsilon \boldsymbol{\rho}_t), \boldsymbol{y}^{(i)})$
    (On a truly quadratic cost function, analytically solve for ϵ^* rather than explicitly searching for it)
    Apply update: $\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t + \epsilon^* \boldsymbol{\rho}_t$
    $t \leftarrow t + 1$
  end while

Conjugate Gradients

- d_t and d_{t-1}, are defined as conjugate if $d_t^\top H(J) d_{t-1} = 0$

$$d_t^\top H d_{t-1} = 0 \tag{10}$$

- **Can we calculate the conjugate directions without resorting to these calculations?**

Two directions, d_t and d_{t-1}, are defined as conjugate if $d_t^\top H(J) d_{t-1} = 0$

$$d_t^\top H d_{t-1} = 0 \tag{11}$$

Conjugate Gradients

Two popular methods for computing the β_t:

- **Fletcher-Reeves**:

$$\beta_t = \frac{\nabla_\theta J(\theta_t)^\theta \nabla_\theta J(\theta_t)}{\nabla_\theta J(\theta_{t-1})^\theta \nabla_\theta J(\theta_{t-1})} \tag{12}$$

- **Polak-Ribière**:

$$\beta_t = \frac{(\nabla_\theta J(\theta_t) - \nabla \theta J(\theta_{t-1}))^\top \nabla \theta J(\theta_t)}{\nabla \theta J(\theta_{t-1})^\top \nabla \theta J(\theta_{t-1})} \tag{13}$$

BFGS

- The *Broyden–Fletcher–Goldfarb–Shanno* (BFGS): bring some of the advantages of Newton's method without the computational burden
- BFGS is similar to CG
- Newton's update is given by

$$\theta^* = \theta_0 - H - 1\nabla\theta J(\theta_0), \tag{14}$$

where H is the Hessian of J with respect to θ evaluated at θ_0

$\rightarrow$ approximate the inverse with a matrix $\mathbf{M}_t$ that is iteratively refined by low rank updates to become a better approximation of H^{-1}

- Unlike CG, the success of BFGS is not heavily dependent on the line search finding a point very close to the true minimum along the line